logos/logo_uca.png

logos/logo_est.png

**Université Cadi Ayyad**

**École Supérieure De Technologie — Safi**

Département : Informatique | Filière : Génie Informatique

## Rapport de Projet de Fin d'Études

# Modernisation d'une Application de Gestion de Syndic

*Réalisé par :*

M. **EL AZZAM Bilal**

M. **CHAKOUAT Maraoune**

M. **CHAQRI Ahmed**

*Encadré par :*

M. **HAMZANE Ibrahim**

**Année Universitaire : 2025/2026**

# Dédicace

À nos familles, pour leur soutien constant tout au long de notre parcours.

À notre encadrant, M. **HAMZANE Ibrahim**, pour sa disponibilité et ses conseils.

À tous ceux qui nous ont accompagnés, de près ou de loin, dans la réalisation de ce projet.

# Remerciements

Nous tenons à remercier en premier lieu notre encadrant, M. **HAMZANE Ibrahim**, pour son suivi rigoureux et ses conseils tout au long de ce projet.

Nous remercions également l'École Supérieure de Technologie de Safi pour la qualité de la formation qu'elle nous a offerte, ainsi que l'ensemble du corps enseignant du département informatique.

Enfin, nous exprimons notre gratitude à nos familles pour leur soutien et leur patience durant cette période de travail intensif.

# Résumé

Ce projet porte sur la modernisation d'une application de gestion de syndic. L'ancienne solution, développée sous Microsoft Access, était limitée à un seul poste de travail et ne permettait pas un accès à distance.

Nous avons conçu et développé une application web complète. Elle est accessible depuis n'importe quel navigateur, sans installation nécessaire. L'application propose deux espaces distincts : un tableau de bord complet pour le syndic, et un portail personnel pour chaque copropriétaire.

Le syndic peut gérer les immeubles, les appartements, les copropriétaires, les charges, les réunions et les paiements. Le copropriétaire peut consulter ses charges, effectuer des paiements, répondre aux invitations et recevoir des notifications en temps réel.

L'application est construite avec **Next.js 16**, **Supabase** et **Drizzle ORM**. Elle supporte trois langues (français, anglais et arabe) et génère des avis de paiement au format PDF.

**Mots-clés :** Application web, gestion de syndic, copropriété, Next.js, Supabase, TypeScript, React.

# Table des matières

# Table des figures

# Introduction

La gestion d'une copropriété demande de la rigueur et une bonne organisation. Le syndic doit gérer plusieurs tâches en même temps : enregistrer des propriétaires, suivre les paiements, organiser des réunions et communiquer avec les copropriétaires.

Jusqu'à présent, ces tâches étaient gérées avec une application développée sous Microsoft Access. Cet outil fonctionne uniquement sur un ordinateur de bureau. Il ne permet pas de travailler à distance, ni de collaborer avec plusieurs utilisateurs simultanément.

Notre projet vise à remplacer cette solution par une application web moderne. Elle centralise toutes les opérations dans une seule interface accessible depuis n'importe quel navigateur. Elle propose deux espaces distincts selon le rôle de l'utilisateur.

Ce rapport présente les différentes étapes du projet. Le premier chapitre décrit le contexte et les objectifs. Le deuxième analyse les besoins. Le troisième présente la conception. Le quatrième montre la réalisation. Le cinquième conclut le travail effectué.

CHAPITRE 1

# Présentation Générale

## 1.1 Contexte du projet

La gestion de copropriété repose sur de nombreux processus administratifs et financiers. Les syndics doivent enregistrer des propriétaires, gérer des charges, organiser des assemblées générales et suivre les paiements.

L'application existante a été développée sous Microsoft Access. C'est un système de gestion de bases de données relationnel de Microsoft. Il permet de créer des formulaires, des requêtes et des rapports. Cependant, son architecture est locale. Il ne fonctionne que sur un seul poste de travail, sans possibilité d'accès multi-utilisateurs ni d'accès à distance.

**Figure 1.1** – *Logo de Microsoft Access*

Face à ces limites, il était nécessaire de concevoir une solution web complète. Cette solution doit être accessible depuis tout appareil, gérer plusieurs immeubles et plusieurs rôles utilisateurs, et proposer des fonctionnalités modernes comme les notifications en temps réel et la génération de PDF.

## 1.2 Problématique

L'application actuelle pose plusieurs problèmes concrets. Elle est inaccessible depuis un autre poste que celui où elle est installée. Elle ne permet pas à plusieurs utilisateurs de travailler en même temps. Elle ne propose pas d'interface pour les copropriétaires.

La problématique du projet est la suivante : *comment moderniser l'application de gestion de syndic pour améliorer son accessibilité, sa fiabilité et l'expérience de tous ses utilisateurs ?*

## 1.3 Objectifs du projet

Les objectifs principaux de ce projet sont :

- Créer une application web accessible depuis n'importe quel navigateur, sans installation.
- Proposer deux espaces distincts : un tableau de bord pour le syndic et un portail pour le copropriétaire.
- Permettre la gestion de plusieurs immeubles depuis un seul compte syndic.
- Gérer les appartements, les copropriétaires, les charges et les paiements.
- Organiser les assemblées générales avec invitations et suivi des réponses.
- Envoyer des notifications en temps réel sans rechargement de la page.
- Générer des avis de paiement au format PDF.
- Exporter les données en fichiers Excel.
- Supporter trois langues : français, anglais et arabe (avec interface RTL).

## 1.4 Méthodologie adoptée : Méthode Agile

Nous avons choisi la méthode Agile pour organiser notre travail. Cette approche permet de livrer des fonctionnalités par étapes. Chaque étape est planifiée, développée, testée et validée avant de passer à la suivante.

**Figure 1.2** – *Processus de développement Agile*

Le projet a été découpé en six phases : infrastructure, tableau de bord syndic, charges et paiements, réunions, portail copropriétaire, puis tests et déploiement. Cette organisation permet de livrer une version fonctionnelle à chaque phase, avec ajustement continu des priorités.

CHAPITRE 2

# Analyse des Besoins

## 2.1 Présentation de l'application actuelle et ses limites

L'application actuelle, développée sous Microsoft Access, permet aux syndics de gérer certaines tâches administratives et financières des copropriétés. Toutefois, cette solution présente plusieurs limites qui freinent son adoption et son efficacité :

- **Accessibilité limitée** : l'application est uniquement disponible sur un environnement de bureau, empêchant une gestion mobile et à distance.
- **Ergonomie obsolète** : l'interface utilisateur manque d'intuitivité et de modernité, rendant son utilisation complexe pour les nouveaux utilisateurs.
- **Gestion des mises à jour** : l'application ne permet pas d'évoluer facilement et nécessite des interventions manuelles pour ajouter des fonctionnalités ou corriger des erreurs.
- **Absence d'une synchronisation en temps réel** : les utilisateurs ne peuvent pas accéder aux données simultanément depuis plusieurs appareils.
- **Sécurité des données** : Microsoft Access ne propose pas une architecture robuste en matière de sécurité et de gestion des accès, ce qui représente un risque pour les informations sensibles.

Les captures d'écran ci-dessous présentent l'interface de l'application de gestion de syndic réalisée avec Microsoft Access :

**Figure 2.1** – *Liste des propriétaires sous Access*

**Figure 2.2** – *Liste des versements sous Access*

**Figure 2.3** – *Formulaire de recherche d'un propriétaire sous Access*

**Figure 2.4** – *Formulaire pour effectuer un versement sous Access*

## 2.2 Identification des besoins

### 2.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que l'application doit faire.

**Pour le syndic :**

- Gérer plusieurs immeubles depuis un seul compte.
- Créer, modifier et supprimer des copropriétaires.
- Gérer les appartements avec leur surface, étage, type et tantièmes.
- Créer des charges et les assigner à un appartement ou répartir sur tout l'immeuble.
- Envoyer des rappels de paiement par email.
- Générer des avis de paiement en PDF.
- Organiser des assemblées générales et inviter des copropriétaires.
- Consulter l'historique global des paiements avec des filtres.
- Exporter les données (propriétaires, charges, paiements) en fichier Excel.

**Pour le copropriétaire :**

- Consulter son profil personnel et les informations de son appartement.
- Voir la liste de ses charges et leur statut (payé, partiel, impayé).
- Effectuer un paiement, avec support des paiements partiels.
- Télécharger un avis de paiement PDF pour chaque charge.
- Répondre aux invitations de réunion (accepter ou décliner).
- Recevoir des notifications en temps réel.

### 2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels concernent la qualité et les performances de l'application.

- **Accessibilité** : l'application doit fonctionner sur tout navigateur moderne, sur ordinateur, tablette et mobile.
- **Sécurité** : chaque rôle doit accéder uniquement aux données qui le concernent. L'authentification est gérée par JWT. Les tentatives de connexion sont limitées à 5

par minute par adresse IP.

- **Performance** : les pages se chargent rapidement grâce aux React Server Components. Les requêtes base de données sont optimisées avec des index explicites.
- **Fiabilité financière** : tous les paiements sont effectués dans des transactions base de données. En cas d'erreur, aucune donnée partielle n'est sauvegardée.
- **Internationalisation** : l'interface supporte trois langues — français, anglais et arabe. L'arabe active automatiquement l'interface RTL.
- **Traçabilité** : toutes les actions sensibles sont enregistrées dans un journal d'audit immuable.

## 2.3 Étude des solutions existantes

Nous avons analysé plusieurs types de solutions existantes sur le marché.

| Solution | Avantages | Inconvénients |
|---|---|---|
| **SaaS de syndic** | Interface moderne, accès en ligne | Coûteux, peu personnalisable |
| **ERP immobilier** | Fonctionnalités avancées | Complexe, surdimensionné pour les petites copropriétés |
| **Applications mobiles** | Interface intuitive | Ne couvrent pas tous les besoins métier |
| **Microsoft Access** | Gratuit, simple | Local, obsolète, non collaboratif |

Notre projet combine les avantages d'une application web avec les besoins spécifiques d'un syndic marocain. Il est personnalisable, hébergé en ligne, et accessible sans abonnement mensuel coûteux.

CHAPITRE 3

# Conception

## 3.1 Introduction à la modélisation

Nous avons utilisé l'UML (Unified Modeling Language) et les diagrammes C4 pour modéliser les différentes parties du système. Ces approches permettent de visualiser les interactions, de définir les entités, leurs relations, et de planifier l'architecture avant l'implémentation.

Nous avons réalisé plusieurs diagrammes complémentaires pour couvrir les vues fonctionnelles, logiques et techniques du système :

- Les **diagrammes de cas d'utilisation** (vue *Syndic* et vue *Copropriétaire*) qui décrivent les interactions acteur-système
- Le **diagramme de classes** qui modélise la structure de la base de données
- Le **diagramme d'activité** qui illustre le flux de paiement
- Le **diagramme de séquence** qui décrit le flux d'authentification
- Le **diagramme de composants** qui montre l'architecture technique
- Le **schéma d'architecture** qui synthétise les couches principales du déploiement

## 3.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (UML Use Case) décrit les interactions entre les acteurs et le système. Pour améliorer la lisibilité, nous présentons deux vues séparées : une vue *Syndic* et une vue *Copropriétaire*.

L'application dispose de deux acteurs principaux :

- Le **Syndic** : gère l'ensemble de l'administration (bâtiments, appartements, copropriétaires, charges, réunions)
- Le **Copropriétaire** : accède uniquement à ses données personnelles (charges,

paiements, notifications, réunions)

**Figure 3.1** – *Diagramme de cas d'utilisation — Vue Syndic*

**Figure 3.2** – *Diagramme de cas d'utilisation — Vue Copropriétaire*

## 3.3 Diagramme de classes

Le diagramme de classes (UML Class) représente la structure logique de la base de données PostgreSQL. Il met en évidence les entités principales, leurs attributs et leurs relations.

L'application utilise dix tables principales organisées selon le domaine métier :

- **Authentification** : `users`
- **Immeubles** : `buildings`, `apartments`
- **Copropriétaires** : `owners`
- **Finances** : `charges`, `payments`
- **Réunions** : `meetings`, `meeting_invitations`
- **Audit** : `notifications`, `audit_logs`

**Figure 3.3** – *Diagramme de classes — Structure de la base de données PostgreSQL*

**Point critique de conception** : les valeurs calculées `amount_paid`, `remaining_amount` et le `status` d'une charge ne sont *jamais* stockées en base de données. Elles sont calculées dynamiquement à chaque requête par agrégation sur la table `payments` via Drizzle ORM. Cela garantit la cohérence perpétuelle des données financières, même en cas de correction ultérieure de paiements.

## 3.4 Diagramme d'activité — Flux de paiement

Le diagramme d'activité modélise le flux de travail spécifique au processus de paiement d'une charge. Il montre les étapes critiques de validation, de transaction, et d'audit qui doivent se dérouler sans erreur.

Le flux de paiement suit ces étapes :

1. Validation du formulaire côté client (React Hook Form + Zod)
2. Transmission au Server Action
3. Validation côté serveur (Zod) et vérification du rôle JWT
4. Démarrage d'une transaction Drizzle
5. Insertion atomique du paiement
6. Insertion de l'audit log
7. Fin de transaction
8. Retour du succès au client

**Figure 3.4** – *Diagramme d'activité — Flux de paiement d'une charge*

Le point clé est l'utilisation obligatoire de **transactions Drizzle** pour tous les paiements. Aucun état partiel n'est jamais commis à la base de données.

## 3.5 Diagramme de séquence — Authentification

Le diagramme de séquence illustre les interactions chronologiques lors du processus d'authentification. Il trace le flux du JWT et l'injection du rôle utilisateur.

Le flux d'authentification suit cette séquence :

1. L'utilisateur soumet ses identifiants (email + mot de passe)
2. Supabase Auth authentifie l'utilisateur
3. La base de données exécute le **Custom Access Token Hook**
4. Le Hook récupère le rôle (`syndic` ou `proprietaire`) depuis la table `users`
5. Le rôle est injecté dans le JWT en tant que claim `user_role`
6. Le Server Action lit le rôle depuis le JWT (aucune requête DB supplémentaire)
7. L'utilisateur est redirigé vers son tableau de bord (syndic → `/dashboard`, copropriétaire → `/portal`)
8. Le Middleware vérifie le JWT à chaque requête sans interroger la base de données

**Figure 3.5** – *Diagramme de séquence — Flux d'authentification et injection du rôle JWT*

## 3.6 Diagramme de composants — Architecture technique

Le diagramme de composants offre une vue d'ensemble de l'architecture en montrant les interactions entre les différentes couches (client, serveur, cloud) et les services externes.

**Figure 3.6** – *Diagramme de composants — Architecture système complète*

L'architecture suit ces principes :

- **Client (Navigateur)** : Exécute React Client Components, gère les requêtes HTTP et les WebSockets Realtime
- **Middleware (Edge)** : Vérifie le JWT et applique la limitation de débit (5 tentatives/min/IP)
- **Serveur (Next.js)** : Rend les React Server Components, traite les Server Actions, applique les règles métier et gère les transactions
- **Base de données (Supabase PostgreSQL)** : Stocke les données avec RLS, exécute les hooks JWT personnalisés
- **Services externes** : Upstash Redis (rate limiting), Nodemailer (emails), Sentry (monitoring)

## 3.7 Architecture technique

L'application suit une architecture web moderne, organisée en trois couches principales.

**Figure 3.7** – *Architecture technique — Client, Middleware, Serveur, Base de données*

Le navigateur communique avec le middleware Next.js qui vérifie le JWT à chaque requête. Selon le rôle (`syndic` ou `proprietaire`), l'accès est accordé ou refusé avant même que la page se charge. Les données sont stockées dans Supabase avec des politiques RLS qui isolent les données par utilisateur au niveau de la base de données.

## 3.8 Choix technologiques

### 3.8.1 Framework et frontend

| | |
|---|---|
| **Next.js 16** | **Next.js 16** est le framework principal de l'application. Il utilise l'App Router avec les React Server Components. Les pages se chargent côté serveur par défaut, ce qui améliore les performances. Les mutations sont gérées par des Server Actions. |
| **React 19** | **React 19** est la bibliothèque d'interface utilisateur. Les composants client sont utilisés uniquement quand l'interaction navigateur est nécessaire. |
| **TypeScript** | **TypeScript 5** ajoute le typage statique. Il détecte les erreurs avant l'exécution et améliore la qualité du code sur l'ensemble du projet. |
| **Tailwind v4** | **Tailwind CSS v4** gère tous les styles de l'application. Il est utilisé avec **shadcn/ui**, une bibliothèque de composants accessibles et personnalisables. |

### 3.8.2 Base de données et backend

| | |
|---|---|
| **Supabase** | **Supabase** fournit la base de données PostgreSQL, l'authentification et les abonnements temps réel. Un Custom Access Token Hook injecte le rôle de l'utilisateur directement dans le JWT à la connexion. |
| **Drizzle ORM** | **Drizzle ORM** est l'ORM principal. Il génère des requêtes SQL typées avec TypeScript. Toutes les opérations financières s'exécutent dans des transactions Drizzle pour garantir l'intégrité des données. |
| **PostgreSQL** | **PostgreSQL** est le moteur de base de données. Les politiques Row Level Security (RLS) isolent les données de chaque utilisateur directement au niveau base de données. |

### 3.8.3 Fonctionnalités complémentaires

| **Bibliothèque** | **Rôle** |
|---|---|
| TanStack Query v5 | Gestion du cache et revalidation côté client |
| Zustand | État de l'interface (sidebar, état ouvert/fermé) |
| React Hook Form + Zod v4 | Validation des formulaires côté client et serveur |
| next-intl v4 | Internationalisation (FR / EN / AR + RTL) |
| Nodemailer | Envoi d'emails transactionnels via SMTP Gmail |
| @react-pdf/renderer | Génération des avis de paiement en PDF |
| Upstash Redis | Limitation des tentatives de connexion (5/min/IP) |
| SheetJS (xlsx) | Export des données en fichiers Excel |
| Sentry | Suivi des erreurs en production |

### 3.8.4 Déploiement

**Vercel** **Vercel** est une plateforme d'hébergement d'applications web. Dans ce projet, elle permet de publier l'application Next.js sur Internet, d'automatiser le déploiement à chaque mise à jour du code et de rendre l'application accessible en ligne de manière continue.

## 3.9 Outils de travail

**VS Code** **Visual Studio Code** est l'éditeur principal. Il offre une intégration complète avec TypeScript, les extensions ESLint et Prettier, et le support de LaTeX pour la rédaction de ce rapport.

**GitHub** **GitHub** a servi d'hébergement du dépôt et de plateforme de collaboration pour centraliser le code source et suivre l'avancement du projet.

**Git** **Git** a été utilisé pour le versionnement local : suivi de l'historique, création de branches dédiées par fonctionnalité et gestion des commits.

**Drizzle Kit** **Drizzle Kit** génère et applique les migrations de base de données. **Drizzle Studio** offre une interface visuelle pour inspecter les tables et les données en développement.

CHAPITRE 4

# Réalisation

## 4.1 La landing page

La landing page est la page publique de présentation. Elle décrit les fonctionnalités de l'application pour les deux rôles. Un bouton d'accès principal redirige vers la page de connexion.

**Figure 4.1** – *La landing page*

## 4.2 La page de connexion

La page de connexion présente un formulaire simple avec un champ email et un champ mot de passe. Il n'y a pas de sélecteur de rôle. Le serveur lit le rôle depuis le JWT et redirige automatiquement : le syndic vers `/dashboard`, le copropriétaire vers `/portal`.

Un lien "Mot de passe oublié" permet de lancer la procédure de réinitialisation via Supabase Auth.

**Figure 4.2** – *La page de connexion*

## 4.3 Le tableau de bord du syndic

### 4.3.1 Vue d'ensemble

La page d'accueil du syndic affiche plusieurs indicateurs calculés sur l'ensemble des immeubles : nombre de copropriétaires actifs, total des appartements, appartements occupés et vacants, charges actives, et total des paiements collectés.

En dessous des indicateurs, un tableau liste toutes les charges impayées ou en retard. Ces valeurs sont calculées dynamiquement par agrégation sur la table `payments`. Aucune valeur dérivée n'est stockée en base de données.

**Figure 4.3** – *Vue d'ensemble du tableau de bord syndic*

### 4.3.2 Gestion des immeubles

Cette page liste tous les immeubles gérés par le syndic. Le syndic peut créer plusieurs immeubles depuis un seul compte. Chaque immeuble a un nom, une adresse et un total de tantièmes.

**Figure 4.4** – *Page de gestion des immeubles*

### 4.3.3 Gestion des copropriétaires

La page des copropriétaires affiche la liste de tous les copropriétaires actifs. On peut rechercher par nom, filtrer, modifier ou supprimer un copropriétaire.

**Figure 4.5** – *Liste des copropriétaires*

#### *Ajout d'un copropriétaire*

Un clic sur le bouton d'ajout ouvre une fenêtre modale centrée. Le formulaire demande : prénom, nom, email, téléphone, immeuble, appartement et date d'entrée. À la validation, le système crée automatiquement un compte Supabase Auth pour le copropriétaire et lui envoie un email de bienvenue avec ses identifiants.

**Figure 4.6** – *Formulaire d'ajout d'un copropriétaire*

### 4.3.4 Gestion des appartements

Les appartements sont affichés par immeuble dans des sections repliables. Chaque appartement indique son numéro, son étage, sa surface, son type (Studio, T1, T2, T3, T4+), ses tantièmes et son statut d'occupation.

**Figure 4.7** – *Liste des appartements*

#### *Ajout d'un appartement*

Le formulaire d'ajout d'appartement s'affiche dans une boîte de dialogue centrée. Le statut d'occupation se met à jour automatiquement selon l'assignation des copropriétaires.

**Figure 4.8** – *Formulaire d'ajout d'un appartement*

### 4.3.5 Gestion des charges

La page des charges liste toutes les charges créées par le syndic. Pour chaque charge, le statut (payé / partiel / impayé), le montant total, le montant payé et le solde restant sont calculés dynamiquement.

Pour les charges avec un solde restant, deux actions sont disponibles : envoyer un rappel email au copropriétaire ou télécharger un avis de paiement PDF.

**Figure 4.9** – *Liste des charges*

#### *Création d'une charge*

Le formulaire de création s'affiche dans une fenêtre modale centrée et permet de définir le titre, la description, la catégorie, le montant, la date d'échéance et la cible. La cible peut être un appartement spécifique ou tout l'immeuble. Dans ce dernier cas, le montant est distribué proportionnellement aux tantièmes de chaque appartement.

**Figure 4.10** – *Formulaire de création d'une charge*

### 4.3.6 Gestion des réunions

La page des réunions liste toutes les assemblées générales planifiées. On peut consulter le statut (planifiée, complétée, annulée) et le nombre d'invités.

**Figure 4.11** – *Liste des réunions*

#### *Création d'une réunion*

Le formulaire de création s'affiche dans une fenêtre modale centrée et demande le titre, la description, la date, l'heure de début, l'heure de fin et le lieu.

**Figure 4.12** – *Formulaire de création d'une réunion*

#### *Détail d'une réunion*

La page détail affiche toutes les informations de la réunion. En dessous, la liste des copropriétaires invités avec leur statut de réponse (invité, accepté, décliné).

**Figure 4.13** – *Page de détail d'une réunion*

### 4.3.7 Historique des paiements

Cette page affiche l'historique global de tous les paiements. Des filtres permettent de rechercher par date, par copropriétaire et par catégorie de charge. Un résumé en haut indique le total émis, le total collecté et le solde restant.

**Figure 4.14** – *Historique global des paiements*

### 4.3.8 Paramètres du bâtiment

La page paramètres permet au syndic de configurer les informations de l'immeuble sélectionné : nom, adresse et coordonnées de contact. Ces informations apparaissent sur tous les PDF générés par l'application.

**Figure 4.15** – *Page des paramètres de l'immeuble*

## 4.4 Le portail copropriétaire

### 4.4.1 Mon espace

La page d'accueil du copropriétaire affiche trois indicateurs : le solde restant de ses charges, la prochaine réunion et le nombre de notifications non lues. En dessous, ses informations personnelles, les détails de son appartement (avec ses tantièmes) et les informations de l'immeuble.

Le copropriétaire peut modifier son numéro de téléphone directement sur cette page et changer son mot de passe.

**Figure 4.16** – *Page Mon espace du copropriétaire*

### 4.4.2 Mes paiements

#### *Onglet charges*

Cet onglet liste toutes les charges assignées à l'appartement du copropriétaire. Chaque charge affiche son titre, sa catégorie, son montant total, le montant payé, le solde restant et son statut. Un bouton "Télécharger l'avis PDF" est disponible pour chaque charge.

**Figure 4.17** – *Liste des charges du copropriétaire*

#### *Effectuer un paiement*

Un clic sur "Payer" ouvre un formulaire de paiement. Le copropriétaire saisit le montant (les paiements partiels sont supportés), la méthode de paiement (virement, chèque ou espèces) et une référence. La transaction s'exécute dans une transaction base de données. En cas d'erreur, aucune donnée partielle n'est sauvegardée.

**Figure 4.18** – *Formulaire de paiement*

#### *Onglet historique*

L'onglet historique liste tous les paiements effectués par le copropriétaire, avec la date, le montant, la méthode et la référence. Un bouton d'export permet de télécharger l'historique en fichier Excel.

**Figure 4.19** – *Historique des paiements du copropriétaire*

### 4.4.3 Mes réunions

La page des réunions propose deux onglets : les réunions à venir et les réunions passées.

#### *Réunions à venir*

Pour chaque réunion planifiée, le copropriétaire voit le titre, la date, l'heure, le lieu et l'ordre du jour. Si il n'a pas encore répondu, deux boutons lui permettent d'accepter ou de décliner l'invitation.

**Figure 4.20** – *Réunions à venir du copropriétaire*

#### *Réunions passées*

L'onglet des réunions passées est dédié à l'historique des assemblées générales. Sur cette capture, aucune réunion passée n'est encore disponible.

**Figure 4.21** – *Réunions passées du copropriétaire*

### 4.4.4 Notifications

La page des notifications affiche les éléments en ordre chronologique inverse, avec leur titre, leur contenu et leur date relative. Un indicateur du nombre de notifications non lues est affiché, ainsi qu'une action pour tout marquer comme lu.

Les notifications incluent notamment les nouvelles charges, les invitations de réunion et les confirmations de paiement. Les mises à jour arrivent en temps réel via Supabase Realtime, sans rechargement de la page.

**Figure 4.22** – *Page des notifications*

## 4.5 Avis de paiement PDF

L'application génère des avis de paiement formels au format PDF. Ils peuvent être téléchargés par le syndic (depuis la page des charges) et par le copropriétaire (depuis son portail).

Le document contient : les informations du destinataire, les informations de l'immeuble, le détail de la charge et un récapitulatif financier (montant total, montant payé, solde restant). Selon le cas, il peut aussi inclure des instructions de paiement. Les informations de l'immeuble sont lues depuis la table `buildings`, ce qui garantit que les PDF sont toujours à jour.

**Figure 4.23** – *Avis de paiement généré en PDF*

## 4.6 Internationalisation et support RTL

L'application supporte trois langues : le français (par défaut), l'anglais et l'arabe. L'utilisateur peut changer la langue depuis l'icône de langue dans la barre de navigation.

Pour l'arabe, l'interface bascule automatiquement en mode RTL (de droite à gauche). Tous les éléments de la page — textes, icônes, marges, disposition — s'inversent correctement.

**Figure 4.24** – *Interface en arabe avec disposition RTL*

CHAPITRE 5

# Conclusion

Ce projet nous a permis de moderniser une application de gestion de syndic vieillissante et de la transformer en une solution web complète et accessible.

Nous avons remplacé une application locale Microsoft Access par une plateforme web hébergée, accessible depuis n'importe quel navigateur. Le syndic dispose d'un tableau de bord centralisé pour gérer plusieurs immeubles, des copropriétaires, des charges, des réunions et des paiements. Le copropriétaire dispose d'un portail personnel pour suivre ses charges, effectuer des paiements et répondre aux invitations.

Sur le plan technique, ce projet nous a permis de mettre en pratique des technologies modernes : Next.js 16 avec l'App Router, Supabase pour l'authentification et la base de données, Drizzle ORM pour l'accès aux données typé, et les React Server Components pour des performances optimales.

Nous avons respecté plusieurs contraintes techniques importantes : les valeurs financières dérivées ne sont jamais stockées en base, les suppressions sont toujours logiques (soft delete), les mutations financières s'exécutent dans des transactions, et les rôles sont lus depuis le JWT sans requête base de données.

L'application est déployée en production sur Vercel et accessible en ligne.

Pour la suite, plusieurs améliorations sont envisageables : l'ajout d'un tableau de bord analytique avec des graphiques d'évolution des paiements, la gestion de documents (procès-verbaux de réunions), et une application mobile pour les copropriétaires.

Ce projet a été une expérience enrichissante. Il nous a permis d'aborder des problématiques réelles de gestion de données, de sécurité et de performance dans un contexte applicatif professionnel.

# Webographie

| **Outil / Technologie** | **Source** |
|---|---|
| Next.js | https://nextjs.org/docs |
| React | https://react.dev |
| TypeScript | https://www.typescriptlang.org/docs |
| Tailwind CSS | https://tailwindcss.com/docs |
| shadcn/ui | https://ui.shadcn.com |
| Supabase | https://supabase.com/docs |
| Drizzle ORM | https://orm.drizzle.team/docs |
| TanStack Query | https://tanstack.com/query/latest/docs |
| Zustand | https://docs.pmnd.rs/zustand/getting-started/introduction |
| Zod | https://zod.dev |
| React Hook Form | https://react-hook-form.com |
| Nodemailer | https://nodemailer.com/about |
| @react-pdf/renderer | https://react-pdf.org |
| Upstash Redis | https://upstash.com/docs/redis |
| Sentry | https://docs.sentry.io/platforms/javascript/guides/nextjs |
| next-intl | https://next-intl-docs.vercel.app |
| SheetJS | https://sheetjs.com |
| Vercel | https://vercel.com/docs |
| PlantUML | https://plantuml.com |
| Devicon | https://devicon.dev |